同步加载优化之 FlushAsyncLoading

一、前言

项目现状

- **同步加载太多**

项目开发早期，大家对资源加载还未建立规范，清一色全是同步加载，尽管经过一些优化，还是存在很多同步加载代码

- **预加载**

为了减少游戏内卡顿，加载完关卡时会预加载较多资源；同时为了减少玩家进游戏的等待时间，预加载资源都采用异步加载，且不会等待所有资源加载完毕玩家就可以进入地图

- **性能问题**

预加载会申请很多异步加载请求，如果此时，Gameplay 层来一个同步加载，就会导致比较严重的 BigJunk（后续会分析具体原因）

二、引擎底层剖析

引擎底层处理资源加载时，不管是同步还是异步，都走的异步加载，二者处理的区别在于：

2.1 异步加载

在异步线程(AsyncLoadingThread)处理资源加载任务，不阻塞主线程(GameThread)

2.2 同步加载

同步加载依然是发起异步加载请求

C/C++

```
UPackage* LoadPackageInternal(UPackage* InOuter, const TCHAR* InLongPackageNameOrFilename, uint32 LoadFlags, FLinkerLoad* ImportLinker, FArchive* InReaderOverride, const FLinkerInstancingContext* InstancingContext)
{
    ......

    UPackage* Result = nullptr;

    if (FPlatformProperties::RequiresCookedData() && GEventDrivenLoaderEnabled
        && EVENT_DRIVEN_ASYNC_LOAD_ACTIVE_AT_RUNTIME
        )
    {
        ......

        FName PackageFName(*InPackageName);

        {
            if (FCoreDelegates::OnSyncLoadPackage.IsBound())
            {
                FCoreDelegates::OnSyncLoadPackage.Broadcast(InName);
            }

            int32 RequestID = LoadPackageAsync(InName, nullptr, *InPackageName);

            if (RequestID != INDEX_NONE)
            {
                FlushAsyncLoading(RequestID);
            }
        }

        Result = (InOuter ? InOuter : FindObjectFast<UPackage>(nullptr, PackageFName));
        return Result;
    }

```

```
    ......
}
```

但同步加载会阻塞 GameThread，直到资源加载完毕

C/C++

```cpp
void FAsyncLoadingThread::FlushLoading(int32 PackageID)
{
     if (IsAsyncLoading())
    {
        // Flushing async loading while loading is suspend will result in infinite stall
        UE_CLOG(IsAsyncLoadingSuspendedInternal(), LogStreaming, Fatal, TEXT("Cannot Flush Async Loading while async loading is suspended (%d)"), GetAsyncLoadingSuspendedCount());

        SCOPE_CYCLE_COUNTER(STAT_FAsyncPackage_FlushAsyncLoadingGameThread);

        if (PackageID != INDEX_NONE && !ContainsRequestID(PackageID))
        {
            return;
        }

        FCoreDelegates::OnAsyncLoadingFlush.Broadcast();

        double StartTime = FPlatformTime::Seconds();

        // Flush async loaders by not using a time limit. Needed for e.g. garbage collection.
        {
            TUniquePtr<FFlushTree> FlushTree;
            if (PackageID != INDEX_NONE)
            {
                FlushTree = MakeUnique<FFlushTree>(PackageID);
            }
            SCOPE_CYCLE_COUNTER(STAT_FAsyncPackage_TickAsyncLoadingGameThread);
            while (IsAsyncLoading())
            {
                EAsyncPackageState::Type Result = TickAsyncLoading(false, false, 0, FlushTree.Get());
                if (PackageID != INDEX_NONE && !ContainsRequestID(PackageID))
```

```
                {
                    break;
                }

                if (IsMultithreaded())
                {
                    // Update the heartbeat and sleep. If we're not multithreading, the heartbeat is updated after each package has been processed
                    FThreadHeartBeat::Get().HeartBeat();
                    FPlatformProcess::SleepNoStats(0.0001f);
                }

                // push stats so that we don't overflow number of tags per thread during blocking loading
                LLM_PUSH_STATS_FOR_ASSET_TAGS();
            }
        }
        ......
    }
}
```

2.3 同步队列 QueuedPackages

引擎底层在处理同步和异步加载时，会维护一个同步队列 QueuedPackages，然后在异步线程中挨个处理每个加载请求

C/C++

```cpp
void FAsyncLoadingThread::QueuePackage(FAsyncPackageDesc& Package)
{
    {
#if THREADSAFE_UOBJECTS
        FScopeLock QueueLock(&QueueCritical);
#endif
#if !UE_BUILD_SHIPPING
        if(CheckForFilePackageOpenLogCommandLine())
        {
            FPlatformFileOpenLog* PlatformFileOpenLog = (FPlatformFileOpenLog*)(FPlatformFileManager::Get().FindPlatformFile(FPlatformFileOpenLog::GetTypeName()));
            if (PlatformFileOpenLog != nullptr)
            {
                PlatformFileOpenLog->AddPackageToOpenLog(*Package.Name.ToString());
            }
        }
#endif

        QueuedPackagesCounter.Increment();
        QueuedPackages.Add(new FAsyncPackageDesc(Package, MoveTemp(Package.PackageLoadedDelegate)));
    }
    NotifyAsyncLoadingStateHasMaybeChanged();

    QueuedRequestsEvent->Trigger();
}
```

C/C++

```cpp
uint32 FAsyncLoadingThread::Run()
{
    LLM_SCOPE(ELLMTag::AsyncLoading);

    AsyncLoadingThreadID = FPlatformTLS::GetCurrentThreadId();

    TRACE_LOADTIME_START_ASYNC_LOADING();

    if (!IsInGameThread())
    {
        FPlatformProcess::SetThreadAffinityMask(FPlatformAffinity::GetAsyncLoadingThreadMask());
        FMemory::SetupTLSCachesOnCurrentThread();
    }

    bool bWasSuspendedLastFrame = false;
    while (StopTaskCounter.GetValue() == 0)
    {
        if (IsLoadingSuspended.GetValue() == 0)
        {
            if (bWasSuspendedLastFrame)
            {
                bWasSuspendedLastFrame = false;
                ThreadResumedEvent->Trigger();
            }
            if (!IsGarbageCollectionWaiting())
            {
                bool bDidSomething = false;
                // 异步线程AsyncLoadingThread中Tick
                TickAsyncThread(true, false, 0.033f, bDidSomething);
            }
        }
        else if (!bWasSuspendedLastFrame)
        {
            bWasSuspendedLastFrame = true;
            ThreadSuspendedEvent->Trigger();
        }
        else
```

```
38         {
39             FPlatformProcess::SleepNoStats(0.001f);
40         }
41     }
42     return 0;
43 }
```

C/C++

```
int32 FAsyncLoadingThread::CreateAsyncPackagesFromQueue(bool bUseTimeLimit, bool bUseFullTimeLimit, float TimeLimit, FFlushTree* FlushTree)
{
    SCOPED_LOADTIMER(CreateAsyncPackagesFromQueueTime);

    FAsyncLoadingTickScope InAsyncLoadingTick(*this);

    int32 NumCreated = 0;
    checkSlow(IsInAsyncLoadThread());

    int32 TimeSliceGranularity = 1; // do 4 packages at a time

    if (!bUseTimeLimit)
    {
        TimeSliceGranularity = MAX_int32; // no point in taking small steps
    }

    TArray<FAsyncPackageDesc*> QueueCopy;
    double TickStartTime = FPlatformTime::Seconds();
    do
    {
        {
#if THREADSAFE_UOBJECTS
            FScopeLock QueueLock(&QueueCritical);
#endif
            QueueCopy.Reset();
            QueueCopy.Reserve(FMath::Min(TimeSliceGranularity, QueuedPackages.Num()));

            int32 NumCopied = 0;

            for (FAsyncPackageDesc* PackageRequest : QueuedPackages)
            {
                if (NumCopied < TimeSliceGranularity)
                {
                    NumCopied++;
                    QueueCopy.Add(PackageRequest);
```

```
                }
                else
                {
                    break;
                }
            }
            if (NumCopied)
            {
                QueuedPackages.RemoveAt(0, NumCopied, false);
            }
            else
            {
                break;
            }
        }

        if (QueueCopy.Num() > 0)
        {
            SCOPE_CYCLE_COUNTER(STAT_FAsyncPackage_CreateAsyncPackagesFromQueue);
            double Timer = 0;
            {
                SCOPE_SECONDS_COUNTER(Timer);
                // 挨个处理QueuedPackages中的异步加载请求
                for (FAsyncPackageDesc* PackageRequest : QueueCopy)
                {
                    ProcessAsyncPackageRequest(PackageRequest, nullptr, FlushTree);
                    delete PackageRequest;
                }
            }
            UE_LOG(LogStreaming, Verbose, TEXT("Async package requests inserted in %fms"), Timer * 1000.0);
        }

        NumCreated += QueueCopy.Num();
    } while(!IsTimeLimitExceeded(TickStartTime, bUseTimeLimit, TimeLimit, TEXT("CreateAsyncPackagesFromQueue")));

```

```
    return NumCreated;
}
```

2.4 优先队列 EventQueue

每个资源加载会经历多个加载事件，每个事件也代表资源加载的某一阶段：

C/C++

```
    /** [EDL] Queues CreateLinker event */
    void QueueEvent_CreateLinker(FAsyncPackage* Pkg, int32 EventSystemPriority = 0);
    /** [EDL] Queues FinishLinker event */
    void QueueEvent_FinishLinker(FWeakAsyncPackagePtr WeakPtr, int32 EventSystemPriority = 0);
    /** [EDL] Queues StartImportPackages event */
    void QueueEvent_StartImportPackages(FAsyncPackage* Pkg, int32 EventSystemPriority = 0);
    /** [EDL] Queues SetupImports event */
    void QueueEvent_SetupImports(FAsyncPackage* Pkg, int32 EventSystemPriority = 0);
    /** [EDL] Queues SetupExports event */
    void QueueEvent_SetupExports(FAsyncPackage* Pkg, int32 EventSystemPriority = 0);
    /** [EDL] Queues ProcessImportsAndExports event */
    void QueueEvent_ProcessImportsAndExports(FAsyncPackage* Pkg, int32 EventSystemPriority = 0);
    /** [EDL] Queues ExportsDone event */
    void QueueEvent_ExportsDone(FAsyncPackage* Pkg, int32 EventSystemPriority = 0);
    /** [EDL] Queues ProcessPostload event */
    void QueueEvent_ProcessPostloadWait(FAsyncPackage* Pkg, int32 EventSystemPriority = 0);
    /** [EDL] Queues StartPostLoad event */
    void QueueEvent_StartPostLoad(FAsyncPackage* Pkg, int32 EventSystemPriority = 0);
```

每个资源必须按照上述过程严格执行结束，才能代表该资源加载完毕

所有的资源加载事件会由一个称为 **EventQueue** 的数据结构来维护加载事件的处理顺序，本质上就是个优先队列（大根堆），底层通过 TArray 的 **HeapPush** 贺 **HeapPop** 来维护大根堆特性，保证按序处理资源加载的所有事件

C/C++

```
/** [EDL] Event queue for the event driven loader */
struct FAsyncLoadEventQueue
{
    //FCriticalSection AsyncLoadEventQueueCritical; //@todoio maybe this doesn't need to be protected by a critical section.
    int32 RunningSerialNumber;
    TArray<FAsyncLoadEvent> EventQueue;

    FAsyncLoadEventQueue()
        : RunningSerialNumber(0)
    {
    }

    FORCEINLINE bool IsEmpty() const
    {
        return EventQueue.Num() <= 0;
    }

    FORCEINLINE_DEBUGGABLE void AddAsyncEvent(EAsyncLoadEventType EventType, int32 UserPriority, int32 PackageSerialNumber, int32 EventSystemPriority, TFunction<void(FAsyncLoadEventArgs& Args)>&& Payload)
    {
        //FScopeLock Lock(&AsyncLoadEventQueueCritical);
        //@todoio check(FAsyncLoadingThread::IsInAsyncLoadThread());
        EventQueue.HeapPush(FAsyncLoadEvent(EventType, UserPriority, PackageSerialNumber, EventSystemPriority, ++RunningSerialNumber, Forward<TFunction<void(FAsyncLoadEventArgs& Args)>>(Payload)));
    }

    bool PopAndExecute(FAsyncLoadEventArgs& Args)
    {
        FAsyncLoadEvent Event;
        bool bResult = false;
        {
            //FScopeLock Lock(&AsyncLoadEventQueueCritical);
            //@todoio check(FAsyncLoadingThread::IsInAsyncLoadThread());
            if (EventQueue.Num())
```

```
            {
                EventQueue.HeapPop(Event, false);
                bResult = true;
            }
        }
        if (bResult)
        {
            Event.Payload(Args);
        }
        return bResult;
    }
};
```

单个加载事件会根据以下几个属性排序

UserPriority：资源加载优先级

EventSystemPriority：资源事件的系统层优先级

PackageSerialNumber：资源加载序号

SerialNumber：当前资源事件序号

C/C++

```cpp
/** [EDL] Event Driven loader event */
struct FAsyncLoadEvent
{
    ......

    FORCEINLINE_DEBUGGABLE bool operator<(const FAsyncLoadEvent& Other) const
    {
        if (UserPriority != Other.UserPriority)
        {
            return UserPriority > Other.UserPriority;
        }
        if (EventSystemPriority != Other.EventSystemPriority)
        {
            return EventSystemPriority > Other.EventSystemPriority;
        }
        if (PackageSerialNumber != Other.PackageSerialNumber)
        {
            return PackageSerialNumber > Other.PackageSerialNumber; // roughly DFS
        }
        return SerialNumber < Other.SerialNumber;
    }
}
```

2.5 同步加载会造成 BigJunk 的原因

当 Package 走到 QueueEvent_StartPostLoad 阶段后，会被加入到 AsyncPackagesReadyForTick

C/C++

```
void FAsyncPackage::Event_StartPostload()
{
    LLM_SCOPE(ELLMTag::AsyncLoading);

    Linker->GetAsyncLoader()->LogItem(TEXT("Event_StartPostload"));
    check(AsyncPackageLoadingState == EAsyncPackageLoadingState::ReadyFor
PostLoad);
    check(!PackagesIMayBeWaitingForBeforePostload.Num());
    check(!PackagesIAmWaitingForBeforePostload.Num());
    check(!OtherPackagesWaitingForMeBeforePostload.Num());
    AsyncPackageLoadingState = EAsyncPackageLoadingState::PostLoad_Etc;
    EventDrivenLoadingComplete();
    {
        FUObjectSerializeContext* LoadContext = GetSerializeContext();
        LoadContext->ReserveObjectsLoaded(LoadContext->GetNumObjectsLoa
ded() + Linker->ExportMap.Num());
        for (int32 LocalExportIndex = 0; LocalExportIndex < Linker->Exp
ortMap.Num(); LocalExportIndex++)
        {
            FObjectExport& Export = Linker->ExportMap[LocalExportInde
x];
            UObject* Object = Export.Object;
            checkSlow(!(Object && !ReferencedObjects.Contains(Objec
t)));
            if (Object && (Object->HasAnyFlags(RF_NeedPostLoad) || Link
er->bDynamicClassLinker || Object->HasAnyInternalFlags(EInternalObjectF
lags::AsyncLoading)))
            {
                check(Object->IsValidLowLevelFast());
                LoadContext->AddLoadedObject(Object);
            }
        }
    }
    check(!AsyncLoadingThread.AsyncPackagesReadyForTick.Contains(thi
s));
    AsyncLoadingThread.AsyncPackagesReadyForTick.Add(this);
}
```

后续会统一处理 AsyncPackagesReadyForTick 内所有的 Package 的 PostLoad 逻辑

C/C++

```
EAsyncPackageState::Type FAsyncLoadingThread::ProcessAsyncLoading(int32 & OutPackagesProcessed, bool bUseTimeLimit /*= false*/, bool bUseFullTimeLimit /*= false*/, float TimeLimit /*= 0.0f*/, FFlushTree* FlushTree)
{
    ......

    if (GEventDrivenLoaderEnabled)
    {
#if !UE_BUILD_SHIPPING && !UE_BUILD_TEST
        FScopedRecursionNotAllowed RecursionGuard(*this);
#endif

        FAsyncLoadingTickScope InAsyncLoadingTick(*this);
        uint32 LoopIterations = 0;

        while (true)
        {
            if (bNeedsHeartbeatTick && (++LoopIterations) % 32 == 31)
            {
                // Update heartbeat after 32 events
                FThreadHeartBeat::Get().HeartBeat();
            }

            bool bDidSomething = false;
            {
                bDidSomething = GetPrecacheHandler().ProcessIncoming();
                OutPackagesProcessed += (bDidSomething ? 1 : 0);

                if (IsTimeLimitExceeded(TickStartTime, bUseTimeLimit, TimeLimit, TEXT("ProcessIncoming"), nullptr))
                {
                    return EAsyncPackageState::TimeOut;
                }
            }

            if (IsAsyncLoadingSuspendedInternal())
            {
                return EAsyncPackageState::TimeOut;
```

```
            }

            {
                const float RemainingTimeLimit = FMath::Max(0.0f, TimeLimit - (float)(FPlatformTime::Seconds() - TickStartTime));
                int32 NumCreated = CreateAsyncPackagesFromQueue(bUseTimeLimit, bUseFullTimeLimit, RemainingTimeLimit);
                OutPackagesProcessed += NumCreated;
                bDidSomething = NumCreated > 0 || bDidSomething;
                if (IsTimeLimitExceeded(TickStartTime, bUseTimeLimit, TimeLimit, TEXT("CreateAsyncPackagesFromQueue")))
                {
                    return EAsyncPackageState::TimeOut;
                }
            }
            if (bDidSomething)
            {
                continue;
            }

            {
                FAsyncLoadEventArgs Args;
                Args.bUseTimeLimit = bUseTimeLimit;
                Args.TickStartTime = TickStartTime;
                Args.TimeLimit = TimeLimit;

                Args.OutLastTypeOfWorkPerformed = nullptr;
                Args.OutLastObjectWorkWasPerformedOn = nullptr;
                if (EventQueue.PopAndExecute(Args))
                {
                    OutPackagesProcessed++;
                    if (IsTimeLimitExceeded(Args.TickStartTime, Args.bUseTimeLimit, Args.TimeLimit, Args.OutLastTypeOfWorkPerformed, Args.OutLastObjectWorkWasPerformedOn))
                    {
                        return EAsyncPackageState::TimeOut;
                    }
                    bDidSomething = true;
                }
            }
```

```
            if (bDidSomething)
            {
                continue;
            }
            if (AsyncPackagesReadyForTick.Num())
            {
                SCOPE_CYCLE_COUNTER(STAT_FAsyncLoadingThread_ProcessAsyncLoading);

                OutPackagesProcessed++;
                bDidSomething = true;
                FAsyncPackage* Package = AsyncPackagesReadyForTick[0];
                check(Package->AsyncPackageLoadingState == EAsyncPackageLoadingState::PostLoad_Etc);
                SCOPED_LOADTIMER(ProcessAsyncLoadingTime);

                EAsyncPackageState::Type LocalLoadingState = EAsyncPackageState::Complete;
                if (Package->HasFinishedLoading() == false)
                {
                    float RemainingTimeLimit = FMath::Max(0.0f, TimeLimit - (float)(FPlatformTime::Seconds() - TickStartTime));
                    LocalLoadingState = Package->TickAsyncPackage(bUseTimeLimit, bUseFullTimeLimit, RemainingTimeLimit, FlushTree);
                    if (LocalLoadingState == EAsyncPackageState::TimeOut)
                    {
                        if (IsTimeLimitExceeded(TickStartTime, bUseTimeLimit, TimeLimit, TEXT("TickAsyncPackage")))
                        {
                            return EAsyncPackageState::TimeOut;
                        }
                        UE_LOG(LogStreaming, Error, TEXT("Should not have a timeout when the time limit is not exceeded."));
                        continue;
                    }
                }
                else
                {
                    check(0); // if it has finished loading, it should
```

```
not be in AsyncPackagesReadyForTick
                }
                if (LocalLoadingState == EAsyncPackageState::Complete)
                {
                    {
#if THREADSAFE_UOBJECTS
                        FScopeLock LockAsyncPackages(&AsyncPackagesCritical);
#endif
                        AsyncPackageNameLookup.Remove(Package->GetPackageName());
                        int32 PackageIndex = AsyncPackages.Find(Package);
                        AsyncPackages.RemoveAt(PackageIndex);
                        AsyncPackagesReadyForTick.RemoveAt(0, 1, false); //@todoio this should maybe be a heap or something to avoid the removal cost
                    }

                    // We're done, at least on this thread, so we can remove the package now.
                    AddToLoadedPackages(Package);
                }
                if (IsTimeLimitExceeded(TickStartTime, bUseTimeLimit, TimeLimit, TEXT("TickAsyncPackage")))
                {
                    return EAsyncPackageState::TimeOut;
                }
            }
            ......
        }
        ......
    } // GEventDrivenLoaderEnabled
    else
    {
        ......
    }
}
```

从这里就可以看到，不管是异步加载还是同步加载，都会等待当前所有 Package 加载完毕后，才会统一处理 AsyncPackagesReadyForTick 内的所有 Package，直到走完 PostLoad 逻辑

- **对于异步加载**

不会阻塞主线程，因此等待没什么影响

- **对于同步加载**

会阻塞主线程，因此等待可能会造成很大的影响

如果当前异步加载请求很多，类似我们项目预加载很多资源，同步加载就会等待所有异步加载结束，也就需要等待很长一段时间

> 去年Epic来我们项目交流时，问过他们这个问题，他们给的答案比较模糊：**UE4这套资源加载框架是一位已退休的同事开发的，真实原因很难追溯，他们也不太清楚**
>
> 从我对底层代码的了解来看，和加载框架本身以及UAsset资源结构有很大的关系，处理资源依赖的逻辑也极其复杂，开发这套框架的作者应该是为了安全保障，索性要求同步等待所有异步，否则这里会存在很多难以解决的边界问题
>
> 后续我在解决这个问题的时候也在很大程度上证实了这一点

三、优化

优化的核心思想简单概括一下就是：

同步加载只加载并等待自己及其 Import 依赖，而不是等待当前同步队列所有的 Package

3.1 优化算法

基本的处理算法如下：

- **同步加载优先级**

C/C++

```
enum
{
    EventSystemPriority_MAX = MAX_int32,

    UserPriority0 = 0,

    // [Synchronous load optimization]
    // M = (MAX_int32 >> 1) + OptSyncLoadCounter * SyncLoadPriorityRange
    // N = M + SyncLoadPriorityRange
    // M for sync load package itself
    // [M, N) for sync load import packages
    UserPriority_SyncLoad = (MAX_int32 >> 1),
};
```

[0，UserPriority_SyncLoad) : 表示异步加载

M：代表当前层同步加载 Root

N：代表当前层同步加载的 Import

优先级之所以如此设计是为了解决三大问题：

① 同步加载和异步加载分离

要到做同步加载不等异步加载，必须让二者分离，优先级区分是首要的一步

② 多层级引用

Import 存在多层

③ 同步加载嵌套

例如蓝图 CDO、序列化 Export(Class 为 Import)、PostLoad 阶段触发新的资源加载等

- **DFS 所有 Import**

深度搜索所有未处理的 Import，修改加载优先级

> 同步加载的Import的优先级会随搜索深度增加而增加，保证最底层的Import会优先处理

C/C++

```
void FAsyncPackage::ModifyImportsPriority(FAsyncPackage* Package, const TAsyncLoadPriority NewPriority)
{
    QUICK_SCOPE_CYCLE_COUNTER(STAT_FAsyncPackage_ModifyImportsPriority);

    // Avoid dead recursion
    if (!Package || Package->IsOptSyncLoadOrImport())
    {
        return;
    }

    check(Package->GetOptSyncLoadSerialNum() != INDEX_NONE);
    FOptSyncLoadInfo& OSLI = Package->AsyncLoadingThread.GetOptSyncLoadInfo(Package->GetOptSyncLoadSerialNum());
    if (OSLI.PackagesToBeLoad.Contains(Package))
    {
        return;
    }
    OSLI.PackagesToBeLoad.Add(Package);

    FAsyncLoadingThread& AsyncLoadingThread = Package->AsyncLoadingThread;

    // Modify current package's priority
    Package->SetPriority(NewPriority);
    if (Package->SerialNumber)
    {
        AsyncLoadingThread.ModifyPriority(Package->SerialNumber, NewPriority);
    }

    // This package haven't created linker yet, wait to modify priority when it goes to through 'Event_StartImportPackages'
    FLinkerLoad* Linker = Package->Linker;
    if (!Package->Linker)
    {
        return;
```

```
    }

    // DFS
    for (int32 ImportIndex = 0; ImportIndex< Linker->ImportMap.Num(); ImportIndex++)
    {
        FObjectImport* Import = Package->FindExistingImport(ImportIndex);

        if (Import->XObject && IsFullyLoadedObj(Import->XObject))
        {
            continue; // we already have this thing
        }

        const bool bIsPrestreamRequest = Import->ClassName == PrestreamPackageClassNameLoad;
        if (!GProcessPrestreamingRequests && bIsPrestreamRequest)
        {
            continue;
        }

        // Find the outermost of this import
        if (!Import->OuterIndex.IsNull() && !Import->bImportFailed)
        {
            FObjectImport* ImportOutermost = Import;
            // Here we shouldn't judge 'bImportPackageHandled', just find the outermost
            while (/*!ImportOutermost->bImportPackageHandled && */ImportOutermost->OuterIndex.IsImport())
            {
                ImportOutermost = &Linker->Imp(ImportOutermost->OuterIndex);
            }
            // if (ImportOutermost->bImportPackageHandled)
            // {
            //     continue;
            // }
            check(ImportOutermost->OuterIndex.IsNull() || ImportOutermost->HasPackageName());
            Import = ImportOutermost;
```

```
        }

        if (Import->ClassName != NAME_Package && !bIsPrestreamRequest && !Import->HasPackageName())
        {
            continue;
        }

        // Don't try to import a package that is in an import table that we know is an invalid entry
        if (FLinkerLoad::IsKnownMissingPackage(!Import->HasPackageName() ? Import->ObjectName : Import->GetPackageName()))
        {
            continue;
        }

        bool bCompiledInNotDynamic = false;

        // Find ImportPackage
        FAsyncPackage* ImportPackage = nullptr;
        if (Import->XObject)
        {
            UPackage* ExistingPackage = CastChecked<UPackage>(Import->XObject->GetPackage());
            bCompiledInNotDynamic = IsNativeCodePackage(ExistingPackage);
            ImportPackage = ExistingPackage->LinkerLoad ? ExistingPackage->LinkerLoad->AsyncRoot : nullptr;
        }

        // Compiled in class package don't need to be loaded
        if (bCompiledInNotDynamic)
        {
            continue;
        }

        if (!ImportPackage)
        {
            const FName ImportPackageToLoad = !Import->HasPackageName() ? Import->ObjectName : Import->GetPackageName();
```

```
            const FName ImportPackageFName = Linker->GetInstancingContext().Remap(ImportPackageToLoad);
            ImportPackage = AsyncLoadingThread.FindAsyncPackage(ImportPackageFName);
        }

        if (ImportPackage)
        {
            ImportPackage->SetOptSyncLoadSerialNum(Package->GetOptSyncLoadSerialNum());
        }

        // Modify recursively
        ModifyImportsPriority(ImportPackage, NewPriority + 1);
    }
}
```

- **刷新 EventQueue**

刷新大根堆 EventQueue，保证同步加载插队生效

C/C++

```
/** [EDL] Event queue for the event driven loader */
struct FAsyncLoadEventQueue
{
    //FCriticalSection AsyncLoadEventQueueCritical; //@todoio maybe this doesn't need to be protected by a critical section.
    int32 RunningSerialNumber;
    TArray<FAsyncLoadEvent> EventQueue;
    bool bNeedHeapify;

    FAsyncLoadEventQueue()
        : RunningSerialNumber(0)
        , bNeedHeapify(false)
    {
    }

    FORCEINLINE bool IsEmpty() const
    {
        return EventQueue.Num() <= 0;
    }

    FORCEINLINE_DEBUGGABLE void AddAsyncEvent(EAsyncLoadEventType EventType, int32 UserPriority, int32 PackageSerialNumber, int32 EventSystemPriority, TFunction<void(FAsyncLoadEventArgs& Args)>&& Payload)
    {
        //FScopeLock Lock(&AsyncLoadEventQueueCritical);
        //@todoio check(FAsyncLoadingThread::IsInAsyncLoadThread());
        EventQueue.HeapPush(FAsyncLoadEvent(EventType, UserPriority, PackageSerialNumber, EventSystemPriority, ++RunningSerialNumber, Forward<TFunction<void(FAsyncLoadEventArgs& Args)>>(Payload)));
    }

    bool PopAndExecute(FAsyncLoadEventArgs& Args)
    {
        if (bNeedHeapify)
        {
            QUICK_SCOPE_CYCLE_COUNTER(STAT_AsyncLoadEventQueue_Heapify);
            EventQueue.Heapify();
```

```
            bNeedHeapify = false;
        }

        FAsyncLoadEvent Event;
        bool bResult = false;
        {
            //FScopeLock Lock(&AsyncLoadEventQueueCritical);
            //@todoio check(FAsyncLoadingThread::IsInAsyncLoadThread());
            if (EventQueue.Num())
            {
                EventQueue.HeapPop(Event, false);
                bResult = true;
            }
        }
        if (bResult)
        {
            Event.Payload(Args);
        }
        return bResult;
    }

    void ModifyPriority(int32 PackageSerialNumber, int32 NewPriority)
    {
        for (FAsyncLoadEvent& AsyncLoadEvent : EventQueue)
        {
            if (AsyncLoadEvent.PackageSerialNumber == PackageSerialNumber)
            {
                AsyncLoadEvent.UserPriority = NewPriority;
                bNeedHeapify = true;
            }
        }
    }
};
```

- **StartPostload**

StartPostload 阶段单独处理同步

C/C++

```cpp
void FAsyncPackage::Event_StartPostload()
{
    LLM_SCOPE(ELLMTag::AsyncLoading);

    Linker->GetAsyncLoader()->LogItem(TEXT("Event_StartPostload"));
    check(AsyncPackageLoadingState == EAsyncPackageLoadingState::ReadyForPostLoad);
    check(!PackagesIMayBeWaitingForBeforePostload.Num());
    check(!PackagesIAmWaitingForBeforePostload.Num());
    check(!OtherPackagesWaitingForMeBeforePostload.Num());
    AsyncPackageLoadingState = EAsyncPackageLoadingState::PostLoad_Etc;
    EventDrivenLoadingComplete();
    {
        FUObjectSerializeContext* LoadContext = GetSerializeContext();
        LoadContext->ReserveObjectsLoaded(LoadContext->GetNumObjectsLoaded() + Linker->ExportMap.Num());
        for (int32 LocalExportIndex = 0; LocalExportIndex < Linker->ExportMap.Num(); LocalExportIndex++)
        {
            FObjectExport& Export = Linker->ExportMap[LocalExportIndex];
            UObject* Object = Export.Object;
            checkSlow(!(Object && !ReferencedObjects.Contains(Object)));
            if (Object && (Object->HasAnyFlags(RF_NeedPostLoad) || Linker->bDynamicClassLinker || Object->HasAnyInternalFlags(EInternalObjectFlags::AsyncLoading)))
            {
                check(Object->IsValidLowLevelFast());
                LoadContext->AddLoadedObject(Object);
            }
        }
    }

    AsyncLoadingThread.AddToAsyncPackagesReadyForTick(this, true);

    EventDebugLog(TEXT("Event_StartPostload"));
}
```

```

class FAsyncLoadingThread
{
    ......
    void AddToAsyncPackagesReadyForTick(FAsyncPackage* Pkg, const bool bStartPostLoad = false)
    {
        check(Pkg);

        if (Pkg->IsOptSyncLoadOrImport())
        {
            check(Pkg->GetOptSyncLoadSerialNum() != INDEX_NONE);
            FOptSyncLoadInfo& OSLI = GetOptSyncLoadInfo(Pkg->GetOptSyncLoadSerialNum());

            if (bStartPostLoad)
            {
                check(OSLI.PackagesToBeLoad.Contains(Pkg));
            }

            check(!OSLI.PackagesReadyForTick.Contains(Pkg));
            OSLI.PackagesReadyForTick.Add(Pkg);

            OSLI.UpdateState();

            if (OSLI.IsFinish())
            {
                check(Pkg->IsOptSyncLoad());
            }
        }
        else
        {
            check(!AsyncPackagesReadyForTick.Contains(Pkg));
            AsyncPackagesReadyForTick.Add(Pkg);
        }
    }
    ......
}
```

强制将同步 Package 及其 Import 跳出异步线程，到主线程处理 PostLost 逻辑，从而做到同步不等待异步

C/C++

```
EAsyncPackageState::Type FAsyncLoadingThread::ProcessAsyncLoading(int32 & OutPackagesProcessed, bool bUseTimeLimit /*= false*/, bool bUseFullTimeLimit /*= false*/, float TimeLimit /*= 0.0f*/, FFlushTree* FlushTree)
{
    ......

    if (GEventDrivenLoaderEnabled)
    {
        ......

        while (true)
        {
            ......

            FOptSyncLoadInfo& OSLI = GetOptSyncLoadInfo(OptSyncLoadCounter.GetValue() - 1);
            // Flush opt state for all sync-load
            if (OSLI.IsInProcess())
            {
                OSLI.UpdateState();
            }

            if (!GEnableSyncLoadOptimize || !OSLI.IsFinish())
            {
                {
                    FAsyncLoadEventArgs Args;
                    Args.bUseTimeLimit = bUseTimeLimit;
                    Args.TickStartTime = TickStartTime;
                    Args.TimeLimit = TimeLimit;

                    Args.OutLastTypeOfWorkPerformed = nullptr;
                    Args.OutLastObjectWorkWasPerformedOn = nullptr;

                    if (EventQueue.PopAndExecute(Args))
                    {
                        OutPackagesProcessed++;
                        if (IsTimeLimitExceeded(Args.TickStartTime, Args.bUseTimeLimit, Args.TimeLimit,
```

```
                                                    Args.OutLastTypeOfWorkPerformed, Args.OutLastObjectWorkWasPerformedOn))
                        {
                            return EAsyncPackageState::TimeOut;
                        }
                        bDidSomething = true;
                    }
                }

                if (bDidSomething)
                {
                    continue;
                }
            }

            // If we wanna force flush sync-load packages, don't wait
            if (OSLI.IsFinish() || (!OSLI.IsInProcess() && AsyncPackagesReadyForTick.Num()))
            {
                SCOPE_CYCLE_COUNTER(STAT_FAsyncLoadingThread_ProcessAsyncLoading);

                TArray<FAsyncPackage*>& LocalPackagesReadyForTick = OSLI.IsFinish() ?
                    OSLI.PackagesReadyForTick : AsyncPackagesReadyForTick;

                OutPackagesProcessed++;
                bDidSomething = true;
                FAsyncPackage* Package = LocalPackagesReadyForTick[0];
                check(Package->AsyncPackageLoadingState == EAsyncPackageLoadingState::PostLoad_Etc);
                SCOPED_LOADTIMER(ProcessAsyncLoadingTime);

                EAsyncPackageState::Type LocalLoadingState = EAsyncPackageState::Complete;
                if (Package->HasFinishedLoading() == false)
                {
                    float RemainingTimeLimit = FMath::Max(0.0f, TimeLimit - (float)(FPlatformTime::Seconds() - TickStartTime));
```

```
                    LocalLoadingState = Package->TickAsyncPackage(bUseTimeLimit, bUseFullTimeLimit, RemainingTimeLimit, FlushTree);
                    if (LocalLoadingState == EAsyncPackageState::TimeOut)
                    {
                        if (IsTimeLimitExceeded(TickStartTime, bUseTimeLimit, TimeLimit, TEXT("TickAsyncPackage")))
                        {
                            return EAsyncPackageState::TimeOut;
                        }
                        UE_LOG(LogStreaming, Error, TEXT("Should not have a timeout when the time limit is not exceeded."));
                        continue;
                    }
                }
                else
                {
                    check(0); // if it has finished loading, it should not be in AsyncPackagesReadyForTick
                }

                if (LocalLoadingState == EAsyncPackageState::Complete)
                {
                    {
#if THREADSAFE_UOBJECTS
                        FScopeLock LockAsyncPackages(&AsyncPackagesCritical);
#endif
                        AsyncPackageNameLookup.Remove(Package->GetPackageName());
                        int32 PackageIndex = AsyncPackages.Find(Package);
                        AsyncPackages.RemoveAt(PackageIndex);
                        LocalPackagesReadyForTick.RemoveAt(0, 1, false); //@todoio this should maybe be a heap or something to avoid the removal cost

                        if (OSLI.IsFinish() && !OSLI.PackagesReadyForTick.Num())
                        {
```

```
                                OptSyncLoadCounter.Decrement();
                                OSLI.Reset();
                            }
                        }

                        // We're done, at least on this thread, so we can remove the package now.
                        AddToLoadedPackages(Package);
                    }
                    if (IsTimeLimitExceeded(TickStartTime, bUseTimeLimit, TimeLimit, TEXT("TickAsyncPackage")))
                    {
                        return EAsyncPackageState::TimeOut;
                    }
                }
                ......
            }
            ......
        } // GEventDrivenLoaderEnabled
        else
        {
            ......
        }
}
```

3.2 疑难边界问题

- 多层级引用

需要写一个 DFS 处理算法

> 就是前面列出的FAsyncPackage::ModifyImportsPriority

- 循环引用

例如：A => B => X => ... => Y => A

需要去重：

① 一方面依靠引擎自身在 AddArc 和 ProcessAsyncPackageRequest 内部去重

② 另一方面 FAsyncPackage::ModifyImportsPriority 内部也需要去重 (FOptSyncLoadInfo::PackagesToBeLoad)

- 同步嵌套

需要用一个支持多层同步加载嵌套处理的数据结构

C/C++

```cpp
/**
 * Optimized sync-load info
 */
struct FOptSyncLoadInfo
{
    enum EOptSyncLoadState
    {
        OSLS_None,
        OSLS_InProcess,
        OSLS_Finish,
    };

    /** [EDL] Sync Packages that are ready for tick */
    TArray<FAsyncPackage*> PackagesReadyForTick;

    /** [EDL] Optimized sync-load package count */
    TArray<FAsyncPackage*> PackagesToBeLoad;

    /** [EDL] Optimized sync-load state */
    EOptSyncLoadState State;

    void Reset()
    {
        PackagesReadyForTick.Reset();
        PackagesToBeLoad.Reset();
        State = OSLS_None;
    }

    bool IsValid() const { return State != OSLS_None; }
    bool IsFinish() const { return State == OSLS_Finish; }
    bool IsInProcess() const { return State == OSLS_InProcess; }
    void SetInProcess()
    {
        if (!GEnableSyncLoadOptimize)
        {
            State = OSLS_None;
            return;
        }
```

```

        State = OSLS_InProcess;
    }

    void UpdateState()
    {
        if (!GEnableSyncLoadOptimize) return;

        if (IsFinish() || !IsInProcess()) return;

        const bool bFinish =
            PackagesReadyForTick.Num() > 0 &&
            PackagesReadyForTick[PackagesReadyForTick.Num() - 1]->IsOptSyncLoad();

        if (bFinish)
        {
            State = OSLS_Finish;
        }
    }
};
```

OptSyncLoadCounter：使用线程安全计数器标记当前同步加载所在层级

OptSyncLoadMap：同步加载多层缓存 Map

C/C++

```
    /** [EDL] Sync-load packages' serial number counter */
    FThreadSafeCounter OptSyncLoadCounter;

    /** FOptSyncLoadSerialNum => FOptSyncLoadInfo */
    TArray<FOptSyncLoadInfo> OptSyncLoadMap;
```

- 重复加载

在 ProcessAsyncPackageRequest 内部会处理重复包

C/C++

```
void FAsyncLoadingThread::ProcessAsyncPackageRequest(FAsyncPackageDesc* InRequest, FAsyncPackage* InRootPackage, FFlushTree* FlushTree)
{
    FAsyncPackage* Package = FindExistingPackageAndAddCompletionCallback(InRequest, AsyncPackageNameLookup, FlushTree);

    if (Package)
    {
        if (GEnableSyncLoadOptimize)
        {
            OptUpdateExistingPackage(Package, InRequest);
        }
        else
        {
            // The package is already sitting in the queue. Make sure the its priority, and the priority of all its
            // dependencies is at least as high as the priority of this request
            UpdateExistingPackagePriorities(Package, InRequest->Priority);
        }
    }
    else
    {
        // [BLOCKING] LoadedPackages are accessed on the main thread too, so lock to be able to add a completion callback
#if THREADSAFE_UOBJECTS
        FScopeLock LoadedLock(&LoadedPackagesCritical);
#endif
        Package = FindExistingPackageAndAddCompletionCallback(InRequest, LoadedPackagesNameLookup, FlushTree);
    }

    if (!Package)
    {
        // [BLOCKING] LoadedPackagesToProcess are modified on the main thread, so lock to be able to add a completion callback
#if THREADSAFE_UOBJECTS
```

```
31          FScopeLock LoadedLock(&LoadedPackagesToProcessCritical);
32  #endif
33          Package = FindExistingPackageAndAddCompletionCallback(InRequest, LoadedPackagesToProcessNameLookup, FlushTree);
34      }
35
36      if (!Package)
37      {
38          // New package that needs to be loaded or a package has already been loaded long time ago
39          {
40              // GC can't run in here
41              FGCScopeGuard GCGuard;
42              Package = new FAsyncPackage(*this, *InRequest, EDLBootNotificationManager);
43          }
44          if (InRequest->PackageLoadedDelegate.IsValid())
45          {
46              const bool bInternalCallback = false;
47              Package->AddCompletionCallback(MoveTemp(InRequest->PackageLoadedDelegate), bInternalCallback);
48          }
49          Package->SetDependencyRootPackage(InRootPackage);
50          if (FlushTree)
51          {
52              Package->PopulateFlushTree(FlushTree);
53          }
54
55          // Handle optimized sync-load
56          if (Package->IsOptSyncLoad())
57          {
58              OptCreateSyncLoadInfo(Package);
59          }
60
61          // Add to queue according to priority.
62          InsertPackage(Package, false, EAsyncPackageInsertMode::InsertAfterMatchingPriorities);
63
64          // For all other cases this is handled in FindExistingPackageAndAddCompletionCallback
```

```
        const int32 QueuedPackagesCount = QueuedPackagesCounter.Decrement();
        NotifyAsyncLoadingStateHasMaybeChanged();
        check(QueuedPackagesCount >= 0);
    }
}
```

对于同步加载重复包，有以下四种情况：

InPackage (Old)	InNewRequest	Processible
Async	Async	√
Async	Sync	×
Sync	Async	√
Sync	Sync	×

> ×：当前无法处理的情况
> √：可以处理的情况

第二种情况为何不好处理呢？（第四种情况类似）

即当前同步加载请求已存在，且已是异步加载的 Package，有以下几个原因：

① 加载阶段

此时无法确定这个 Package 具体处于哪个阶段，每个加载阶段需要考虑的情况都很复杂

② DFS

依然需要 DFS 处理 Package 所有的 Import，然后每个 Import 都有可能面临 Package 相同的问题

③ 同步嵌套

由于存在同步嵌套，因此该问题还需要考虑异步加载层级以及同步加载所有的层级，因为 Package 及其 Import 都有可能存在任意层级上

因此不管从哪个方面考虑，这种情况的处理都很复杂

但好在这种情况不多，并且对整体性能影响不大，最简单且稳妥的方式就是直接忽略

> 这种还可以从业务层解决：
> ① 预加载完所有资源后再进游戏，避免同步覆盖异步的情况
> ② 预加载只处理IO较大的资源，小资源在游戏内异步 (这是绝大多数UE项目采用的方案，之前PUBG也是采用类似的策略)
> ③ 遇到同步覆盖异步的情况（我在引擎层加了监控），尽量改为异步

C/C++

```cpp
void FAsyncLoadingThread::OptUpdateExistingPackage(FAsyncPackage* InPackage, FAsyncPackageDesc* InNewRequest)
{
    if (FAsyncLoadEvent::IsOptSyncLoad(InNewRequest->Priority))
    {
        /**
         * There has two situations:
         * ① 'InPackage' is in 'AsyncPackagesReadyForTick'
         *  In this case, there has an async-load task and it has gone through 'Event_StartPostload'.
         *  We can't handle this now for it's hard to redo this package and all its imports, so we
         *  have to wait for it to finish
         *  TODO: Try to solve this problem in the future
         * ② 'InPackage' isn't in 'AsyncPackagesReadyForTick'
         * In this case, we should handle existing package and its imports' priority recursively
         */
        const bool bHasStartPostLoad = AsyncPackagesReadyForTick.Find(InPackage) != INDEX_NONE;
        if (!bHasStartPostLoad && !InPackage->GetLinker())
        {
            // OptCreateSyncLoadInfo(InPackage, false);
            // FAsyncPackage::ModifyImportsPriority(InPackage, InNewRequest->Priority);
        }

        UE_LOG(LogStreaming, Warning, TEXT("FAsyncLoadingThread::OptUpdateExistingPackage Pkg:%s HasStartPostLoad:%d Linker:%d"),
               *InPackage->GetPackageNameToLoad().ToString(), bHasStartPostLoad, !InPackage->GetLinker());
    }
    else
    {
        if (GEventDrivenLoaderEnabled && !InPackage->IsOptSyncLoad())
        {
            InPackage->SetPriority(InNewRequest->Priority);
        }
```

```
    }
}
```

- 偶现 Crash

[PostLoad]

UObject::SetLinker

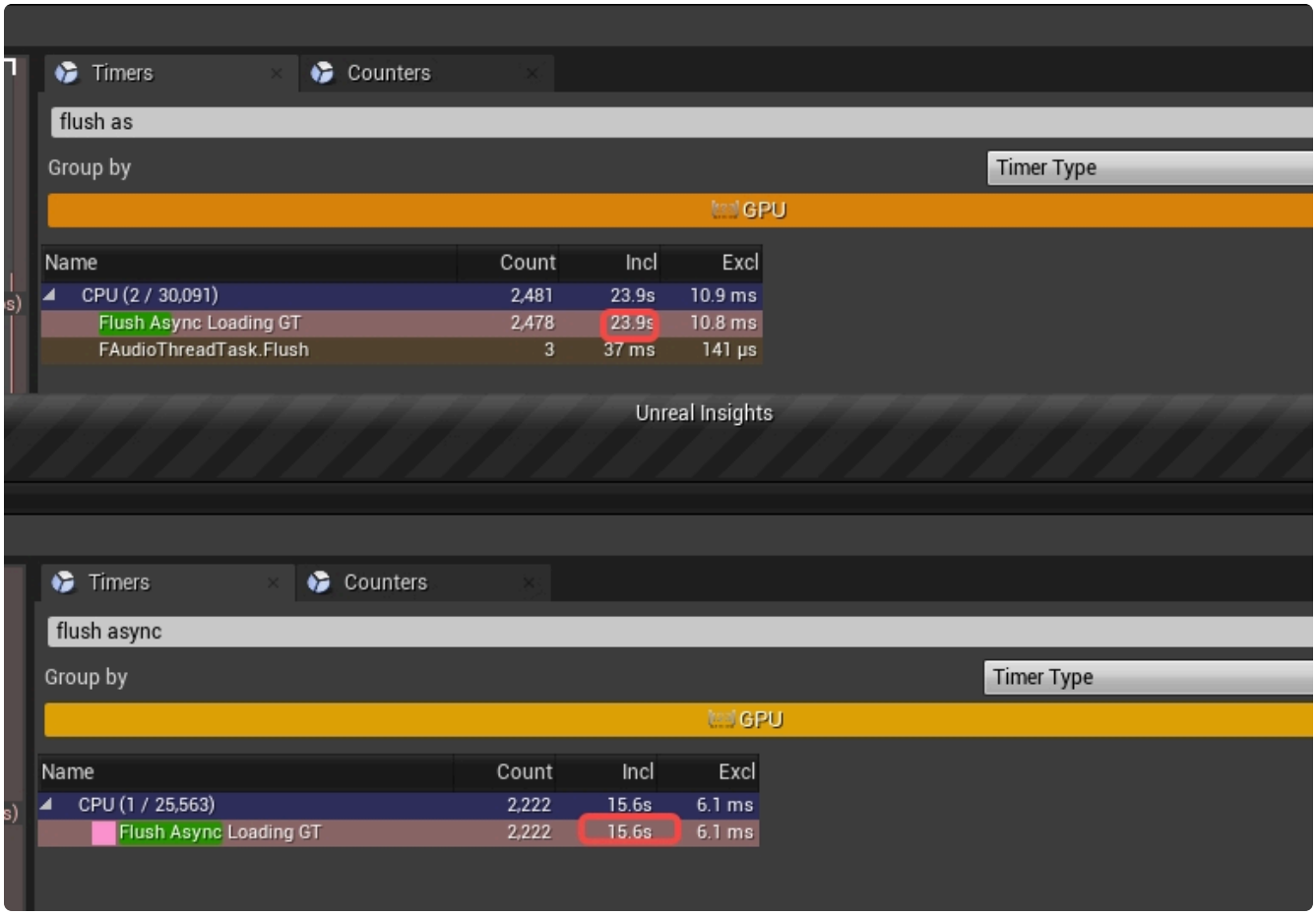
UFunction:Invoke

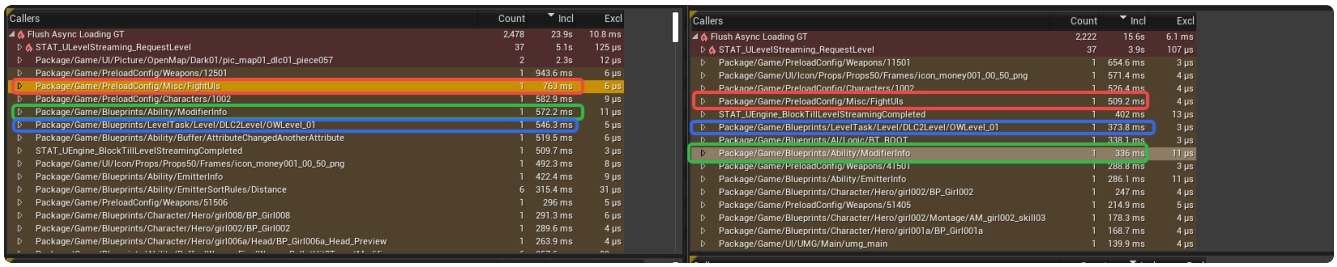
这是最近着手解决的两大问题，主要原因是早期版本未处理同步嵌套和重复加载情况，导致同步会直接覆盖之前的异步，并且这个同步无法正常走完所有加载阶段，偶先会直接跳出异步线程处理 PostLoad，但此时刻的 Export 并未正确加入到 PostLoad 中，也就会导致上面这两种 Crash

> 其实这两个问题也是偶先crash，只不过crash率比较高，所以需要着手解决

> 其实还有一种情况未解决Crash的是
> [Func == nullptr]
> UFunction:Invoke
> 不过很少，大概2000局游戏会出现0~2次，对版本影响不大，以后持续关注

3.3 优化效果





四、总结

推荐的资源加载策略：

- 能用异步尽量用异步
- 预加载只处理 IO 较大的资源，小资源在游戏内异步
- 预加载完所有资源后再进游戏，避免同步覆盖异步的情况

> 性能优化一般应该多考虑从项目入手，引擎优化是保底方案，因为引擎优化通常需要遵循一个“杠杆原理”，就是少许改动获得更大的收益；如果对引擎大动干(肝)戈，一般都会遇到很多难以解决的问题
>
> 像我这次引擎优化一样，几乎将UE资源加载最底层的代码做了深度优化，因此我也花了较多时间，期间还需要依靠珠海QA自动化流水线、不断阅读引擎底层代码以及不断测试Debug等，最终才能得以解决
>
> 其实在业务上层开发的时候，或多或少需要对底层有一定了解，至少要了解到当前开发方式是否存在潜在的性能瓶颈，或许未来的项目可以考虑多加规范一下，**按照引擎规范的方式开发是最稳妥最高效的**